

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Ingeniería en Ciencias y Sistemas
CURSO: *Manejo e Implementación de Archivos*



Nombre:

Carné No.:

ADILZON ALFREDO VELÁSQUEZ HERNÁNDEZ

201908076

Auxiliar: Williams Bramlley Constanza Oscal

Guatemala, 11 de Agosto 2026

1. Arquitectura del sistema

El sistema utiliza una separación entre frontend y backend mediante una API REST. El frontend recibe uno o varios comandos, los prepara y los envía al backend. El backend realiza el análisis léxico y posteriormente la validación sintáctica, sin ejecutar operaciones reales sobre archivos o particiones.

1.1 Flujo de una petición

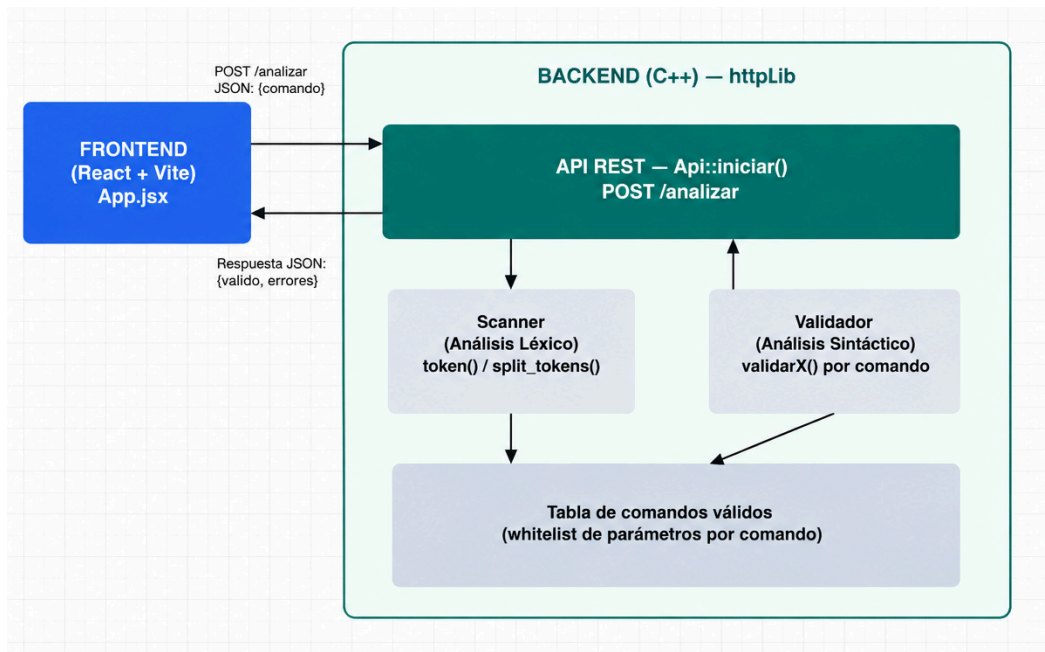
Paso	Descripción
1	El usuario escribe uno o varios comandos en el área de entrada del frontend.
2	El frontend separa el contenido por líneas, ignora comentarios (#) y líneas vacías.
3	Cada línea se envía mediante POST al endpoint /analizar en formato JSON.
4	La clase Api recibe la solicitud y delega el análisis al Scanner.
5	El Scanner obtiene los tokens y el Validador revisa los parámetros del comando.
6	La API construye la respuesta con comando, estado de validez y lista de errores.
7	El frontend muestra el resultado de cada línea al usuario.

Usuario → Frontend (App.jsx) → POST /analizar → Api → Scanner → Validador → JSON → Frontend

1.2 Responsabilidades por capa

Capa	Responsabilidad
Frontend	Entrada de comandos, separación de líneas, envío de solicitudes y presentación de resultados.
API	Recepción de solicitudes, coordinación del análisis y construcción de la respuesta JSON.
Scanner	Reconocimiento léxico y separación del comando en nombre y parámetros.
Validador	Validación de parámetros obligatorios, opcionales, valores permitidos, duplicados y desconocidos.

El sistema sigue una arquitectura de tres capas desacopladas por una API REST, siguiendo el requerimiento técnico del enunciado (backend obligatorio en C++, frontend en un framework moderno, comunicación vía API REST).



El flujo de una petición es el siguiente:

1. El usuario escribe uno o varios comandos en el textarea del frontend (App.jsx).
2. El frontend separa el texto en líneas, descarta comentarios (#) y líneas vacías, y envía cada línea por separado como un POST a /analizar, con el comando en el cuerpo en formato JSON.
3. El backend (clase Api) recibe la petición, invoca al Scanner para tokenizar la línea y luego al Validador correspondiente al comando detectado.
4. El Validador regresa una lista de errores (vacía si el comando es válido). La Api arma la respuesta JSON con los campos comando, valido y errores.
5. El frontend recibe la respuesta y la pinta en pantalla, marcando cada línea como válida o inválida junto con el detalle de errores.

Esta separación permite que el análisis léxico y sintáctico ocurra completamente en el servidor, cumpliendo el requisito de que el sistema no ejecute ninguna acción real sobre archivos ni particiones: el backend solo analiza texto, nunca escribe en disco.

```
#include "../lib/api.h"
#include "../lib/scanner.h"
#include "../lib/validador.h"
#include "../lib/httpLib.h"
#include "../lib/json.hpp"
#include <iostream>
#include <vector>
```

```

using json = nlohmann::json;

using namespace std;

Api::Api() {}

string Api::procesarComando(string comando)
{
    scanner sc;
    Validador validador;
    json respuesta;

    string tk = sc.token(comando);
    string resto = comando;

    if (resto.length() > tk.length())
    {
        resto.erase(0, tk.length() + 1);
    }
    else
    {
        resto = "";
    }

    vector<string> tks = sc.split_tokens(resto);

    vector<string> errores;

    string tkUpper = sc.upper(tk);

    if (sc.compare(tk, "MKDISK")) errores = validador.validarMkdisk(tks);
    else if (sc.compare(tk, "RMDISK")) errores = validador.validarRmdisk(tks);
    else if (sc.compare(tk, "FDISK")) errores = validador.validarFdisk(tks);

```

```

else if (sc.compare(tk, "MOUNT")) errores = validador.validarMount(tks);
else if (sc.compare(tk, "MKFS")) errores = validador.validarMkfs(tks);
else if (sc.compare(tk, "MKUSR")) errores = validador.validarMkusr(tks);
else if (sc.compare(tk, "RMUSR")) errores = validador.validarRmusr(tks);
else if (sc.compare(tk, "MKFILE")) errores = validador.validarMkfile(tks);
else
{
    respuesta["comando"] = tk;
    respuesta["valido"] = false;
    respuesta["errores"] = json::array({"COMANDO INVALIDO \"" + tk + "\""});
    return respuesta.dump();
}

respuesta["comando"] = tkUpper;
respuesta["valido"] = errores.empty();
respuesta["errores"] = errores;
return respuesta.dump();
}

void Api::iniciar(int puerto)
{
    httpplib::Server svr;

    svr.set_default_headers({
        {"Access-Control-Allow-Origin", "*"},
        {"Access-Control-Allow-Methods", "POST, GET, OPTIONS"},
        {"Access-Control-Allow-Headers", "Content-Type"}
    });
}

```

```

svr.Options(".*", [] (const httplib::Request &, httplib::Response &res) {

    res.status = 200;

});

svr.Post("/analizar", [this] (const httplib::Request &req, httplib::Response
&res) {

    try

    {

        json body = json::parse(req.body);

        string comando = body.at("comando").get<string>();

        string resultado = procesarComando(comando);

        res.set_content(resultado, "application/json");

    }

    catch (exception &e)

    {

        json error;

        error["error"] = "JSON invalido o falta el campo 'comando'";

        res.status = 400;

        res.set_content(error.dump(), "application/json");

    }

});

cout << "Servidor escuchando en el puerto " << puerto << "..." << endl;

svr.listen("0.0.0.0", puerto);

}

```

1.3 Responsabilidades por capa

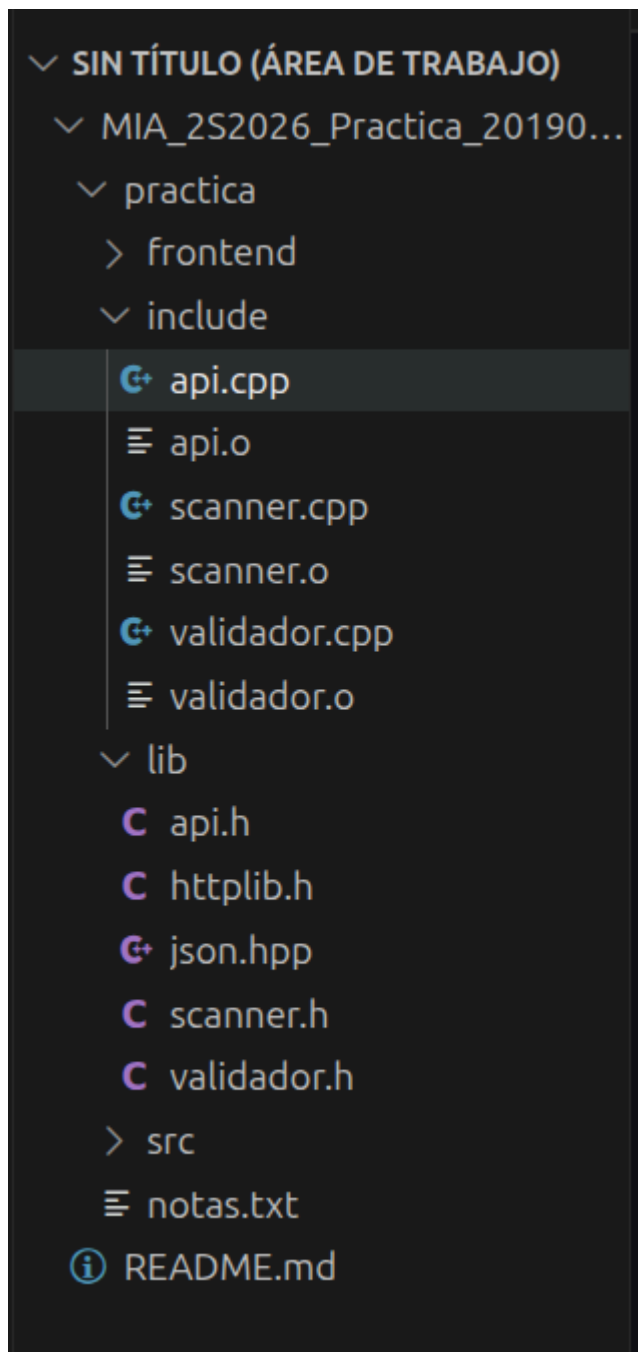
La organización del código separa la interfaz, la API y las responsabilidades del analizador. Esta separación facilita localizar cada componente durante mantenimiento, pruebas o defensa del proyecto.

```

Proyecto/
├── frontend/
│   └── App.jsx
└── backend/
    ├── api.h
    ├── api.cpp
    ├── scanner.h
    ├── scanner.cpp
    ├── validador.h
    └── validador.cpp

```

La representación anterior resume los archivos que aparecen en la implementación documentada. Las carpetas adicionales del repositorio pueden conservarse según la estructura real del proyecto.



1.4 Componentes principales

App.jsx	Interfaz y envío de comandos al backend.
api.h / api.cpp	Endpoint REST y construcción de respuestas.
scanner.h / scanner.cpp	Tokenización de la entrada.
validador.h / validador.cpp	Validación de cada comando y sus parámetros.
Parametro	Struct que representa un parámetro interpretado mediante nombre y valor.

2. Estructuras usadas

El análisis se divide en dos etapas claramente separadas en el código, que corresponden a las dos primeras fases clásicas de un compilador/intérprete: el análisis léxico (lexer) y el análisis sintáctico (parser).

2.1 Analizador léxico (Scanner)

La clase scanner recibe la línea de comando cruda (por ejemplo: `mkdisk -size=10 -path=/home/x.mia`) y la convierte en unidades léxicas (tokens):

- `token(text)`: extrae el nombre del comando (la primera palabra de la línea, por ejemplo MKDISK).
- `split_tokens(text)`: recorre el resto de la línea carácter por carácter, reconociendo cada parámetro en formato `-nombre=valor`, incluyendo casos especiales: valores entre comillas dobles (que pueden contener espacios), parámetros tipo bandera sin valor (como `-r`), y parámetros escritos sin espacio entre ellos.

El resultado de esta etapa es un vector<string> de tokens con forma `nombre=valor`, listo para ser interpretado por la siguiente capa.

Scanner

`token(text)` identifica el nombre del comando, tomando la primera palabra de la línea.

`split_tokens(text)` recorre el resto de la entrada y reconoce parámetros en formato `-nombre=valor`.

Se contemplan valores entre comillas dobles, incluso cuando contienen espacios.

Se reconocen parámetros tipo bandera, como `-r`, que no requieren un valor explícito.

También se soportan parámetros escritos sin espacio entre ellos cuando el analizador puede separarlos correctamente.

Ejemplo conceptual

Entrada:
`mkdisk -size=3000 -unit=K -path=/home/user/Discol.mia`

Comando:
 MKDISK

Parámetros:
`-size = 3000`
`-unit = K`
`-path = /home/user/Discol.mia`

Errores detectables en esta etapa

Comando no reconocido.

Parámetro desconocido.

Parámetros repetidos.

Entrada con formato de parámetro que no puede ser interpretado correctamente.

```
// token(): extrae el lexema del nombre del comando

string tk = token("mkdisk -size=3000 -path=/home/disco.mia");

// tk = "mkdisk"

// split_tokens(): segmenta el resto en lexemas de parámetro

vector<string> tks = split_tokens("-size=3000 -path=/home/disco.mia");

// tks = ["size=3000", "path=/home/disco.mia"]
```

2.2 Analizador sintáctico (Validador)

La clase Validador recibe los tokens producidos por el Scanner y construye una lista de struct Parametro (nombre, valor) mediante parsearParametros(). A partir de ahí aplica, para cada uno de los 8 comandos, una función validarX() que revisa:

- Que los parámetros obligatorios del comando estén presentes.
- Que los valores tengan el formato correcto (numérico, longitud máxima, valores permitidos como BF/FF/WF).
- Que no existan parámetros repetidos ni parámetros que no pertenezcan al comando (mediante validarParametrosConocidos() y la tabla de comandos válidos).

Verificar la presencia de parámetros obligatorios.

Validar que los valores tengan el formato esperado.

Validar valores permitidos como BF, FF, WF, K, M, B, P, E y L, según el comando.

Rechazar parámetros desconocidos.

Rechazar parámetros repetidos.

Aplicar restricciones de longitud o rango cuando corresponda.

```
// validarMkdisk(): valida la gramática propia del comando MKDISK

vector<string> errores = validador.validarMkdisk(tks);

// Reglas aplicadas: -size obligatorio y > 0, -path obligatorio,
// -fit ∈ {BF,FF,WF}, -unit ∈ {K,M}, sin parámetros desconocidos
```

2.3 Tabla de comandos y parámetros válidos

Cada validador define, al inicio, la lista de parámetros que su comando acepta (whitelist). Cualquier parámetro fuera de esa lista —incluyendo abreviaturas como -u en lugar de -unit— se reporta como "Parámetro desconocido".

Comando	Parámetros obligatorios	Parámetros opcionales
MKDISK	-size, -path	-fit (BF/FF/WF), -unit (K/M)
RMDISK	-path	—
FDISK	-size, -path, -name	-unit (B/K/M), -type (P/E/L), -fit (BF/FF/WF)
MOUNT	-path, -name	—
MKFS	-id	-type (Full)
MKUSR	-user, -pass, -grp (máx. 10 car. c/u)	—
RMUSR	-user	—
MKFILE	-path	-r, -size, -cont

```
// =====

// lib/validador.h — declaración

// =====

vector<string> validarConocidos(vector<Parametro> params, vector<string>
permitidos);

// =====

// include/validador.cpp — implementación genérica

// =====

vector<string> Validador::validarConocidos(vector<Parametro> params,
vector<string> permitidos)
{
    vector<string> errores;

    for (Parametro p : params)
    {
        string nombreUp = p.nombre;

        for (char &c : nombreUp) c = toupper(c);

        bool encontrado = false;

        for (string permitido : permitidos)
        {
            string permitidoUp = permitido;
```

```

        for (char &c : permitidoUp) c = toupper(c);

        if (nombreUp == permitidoUp)

        {

            encontrado = true;

            break;

        }

    }

    if (!encontrado)

    {

        errores.push_back("Parametro desconocido: -" + p.nombre);

    }

}

return errores;
}

// =====

// Whitelist declarada al inicio de cada validarXxx()

// (una sola línea nueva por comando, reutilizando validarConocidos)

// =====

// MKDISK — obligatorios: size, path | opcionales: fit, unit
vector<string> desconocidos = validarConocidos(params, {"size", "fit", "unit",
"path"});

// RMDISK — obligatorio: path

```

```

vector<string> desconocidos = validarConocidos(params, {"path"});

// FDISK — obligatorios: size, path, name | opcionales: unit, type, fit
vector<string> desconocidos = validarConocidos(params, {"size", "unit", "path",
"type", "fit", "name"});

// MOUNT — obligatorios: path, name
vector<string> desconocidos = validarConocidos(params, {"path", "name"});

// MKFS — obligatorio: id | opcional: type
vector<string> desconocidos = validarConocidos(params, {"id", "type"});

// MKUSR — obligatorios: user, pass, grp
vector<string> desconocidos = validarConocidos(params, {"user", "pass",
"grp"});

// RMUSR — obligatorio: user
vector<string> desconocidos = validarConocidos(params, {"user"});

// MKFILE — obligatorio: path | opcionales: r, size, cont
vector<string> desconocidos = validarConocidos(params, {"path", "r", "size",
"cont"});

// =====

// Ejemplo de ejecución

// =====

// Entrada: mkdisk -size=3000 -u=K -path=/home/disco.mia

```

```
// "u" no está en {"size","fit","unit","path"} → se reporta como error

vector<string> errores = validador.validarMkdisk(tks);

// errores = ["Parametro desconocido: -u"]
```

2.4 Resumen de responsabilidades por estructura

Estructura	Ubicación	Responsabilidad
class scanner	scanner.h / scanner.cpp	Análisis léxico: separa la línea de comando en el nombre del comando (token()) y en la lista de parámetros nombre=valor (split tokens()).
struct Parametro	validador.h	Representa un parámetro ya interpretado: { nombre, valor }. Es la unidad mínima que produce el analizador léxico y consume el sintáctico.
class Validador	validador.h / validador.cpp	Análisis sintáctico: por cada comando (validarMkdisk, validarFdisk, etc.) valida presencia de obligatorios, formato de valores y rechaza parámetros desconocidos, repetidos o mal formados.
Tabla de comandos válidos	validarParametrosConocidos()	Whitelist de parámetros aceptados por cada comando (ver tabla siguiente). Es la que detecta parámetros abreviados o inventados.
class Api	api.h / api.cpp	Expone el endpoint REST POST /analizar; conecta al scanner y al validador, y devuelve el resultado como JSON.

3. Comandos implementados

El sistema reconoce y valida sintácticamente los 8 comandos definidos en el enunciado de la práctica. Ningún comando ejecuta una acción real sobre archivos o particiones: solo se analiza la estructura léxica y sintáctica del texto ingresado.

1. MKDISK — crea un disco simulado (validación de tamaño, unidad y ajuste).
2. RMDISK — elimina un disco simulado (validación de ruta).
3. FDISK — administra particiones dentro de un disco (crear/tipo/ajuste).
4. MOUNT — monta una partición existente en el sistema.
5. MKFS — formatea una partición montada como EXT2.
6. MKUSR — crea un usuario dentro de la partición.
7. RMUSR — elimina un usuario de la partición.
8. MKFILE — crea un archivo dentro del sistema simulado.

Cada comando cuenta con su propio método `validarX()` en la clase `Validador`, documentado en la sección 2.3, y fue verificado con los ejemplos oficiales del enunciado y con un archivo de prueba adicional proporcionado por el auxiliar de la sección.

4. Ejemplos de uso de los comandos

A continuación se muestra, para cada comando, al menos un ejemplo de entrada válida y uno de entrada inválida, junto con la respuesta que produce el analizador. Todos los ejemplos fueron ejecutados y verificados contra el sistema en funcionamiento.

MKDISK

```
$ mkdisk -Size=3000 -unit=K -path=/home/user/Disco1.mia
```

Válido — comando válido, sin errores.

```
$ mkdisk -size=10 -unit=G -path=/home/x.mia
```

Inválido — El valor de -unit no es válido: "G" (debe ser K o M).

RMDISK

```
$ rmdisk -path="/home/mis discos/Disco4.mia"
```

Válido — comando válido, sin errores.

```
$ rmdisk
```

✗ *Inválido — Falta el parámetro obligatorio -path.*

FDISK

```
$ fdisk -size=1 -type=L -unit=M -fit=BF-path="/mis discos/Disco3.mia"-name="Particion3"
```

Válido — comando válido; el analizador reconoce los tres parámetros aunque estén escritos sin espacio entre ellos.

```
$ fdisk -size=100 -path=/home/x.mia -name=P1 -type=X
```

Inválido — El valor de -type no es válido: "X" (debe ser P, E o L).

MOUNT

```
$ mount -path=/home/Disco2.mia -name=Part2
```

Válido — comando válido, sin errores.

```
$ mount -path=/home/Disco2.mia
```

Inválido — Falta el parámetro obligatorio -name.

MKFS

```
$ mkfs -id=342A
```

Válido — comando válido; al omitir -type se asume Full por defecto.

```
$ mkfs -id=341A -type=quick
```

Inválido — El valor de -type no es válido: "quick" (debe ser Full).

MKUSR

```
$ mkusr -user=user1 -pass=1234 -grp=usuarios
```

Válido — comando válido, sin errores.

```
$ mkusr -user=usuariomuy largo -pass=1234 -grp=usuarios
```

Inválido — El valor de -user excede el máximo de 10 caracteres.

RMUSR

```
$ rmusr -user=user1
```

Válido — comando válido, sin errores.

```
$ rmusr
```

Inválido — Falta el parámetro obligatorio -user.

MKFILE

```
$ mkfile -size=15 -path=/home/user/docs/a.txt -r
```

Válido — comando válido; -r se reconoce como bandera sin valor asignado.

```
$ mkfile -path=/home/x.txt -size=-10
```

Inválido — El valor de -size no puede ser negativo.

Casos adicionales de análisis léxico (aplican a cualquier comando):

Parámetro desconocido

```
$ mkdisk -size=10 -foo=bar -path=/home/x.mia
```

Inválido — Parámetro desconocido: "-foo".

Parámetro repetido

```
$ mkdisk -size=10 -size=20 -path=/home/x.mia
```

Inválido — Parámetro repetido: "-size".

Comando no reconocido

```
$ xyz -size=10
```

Inválido — COMANDO INVALIDO "xyz".